

LAPORAN PRAKTIKUM ANALISIS KOMPLEKSITAS ALGORITMA (BIG O NOTATION)



Disusun Oleh:

NAURA NADHIFA

NIM : 2025573010087

KELAS : TI-1D

JURUSAN : Teknologi Informasi dan Komputer

PRODI : Teknik Informatika

DOSEN PENGAJAR : M. Reza Zulman, SST. M.Sc

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKEUMAWE
2026**

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, makalah praktikum ini dapat diselesaikan dengan baik. Makalah ini disusun sebagai bagian dari kegiatan praktikum pada mata kuliah Struktur Data dan Algoritma yang bertujuan untuk memahami konsep dasar analisis kompleksitas algoritma.

Penulis menyadari bahwa pemahaman mengenai efisiensi algoritma merupakan hal yang sangat penting dalam dunia pemrograman. Oleh karena itu, melalui praktikum ini penulis mencoba mempelajari dan mengimplementasikan konsep Big O Notation, Time Complexity, serta Space Complexity menggunakan bahasa pemrograman JavaScript.

Penulis juga menyadari bahwa makalah ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga makalah ini dapat memberikan manfaat bagi pembaca.

Buketrata, 25 April 2026

Naura Nadhifa

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Praktikum	4
BAB II LANDASAN TEORI.....	5
2.1 Pengertian Big O Notation	5
2.2 Time Complexity	5
2.3 Space Complexity	6
2.4 Perbandingan Algoritma	6
2.5 Optimasi dan Refactoring Algoritma	6
BAB III METODOLOGI PENELITIAN.....	8
3.1 Alat dan Bahan	8
3.2 Langkah-langkah praktikum Function dan Array	8
3.3 Hasil Praktikum	12
3.4 submit ke Github	12
3.5 Analisis Hasil.....	13
BAB IV PENUTUP	14
4.1 Kesimpulan.....	14
4.2 Saran	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, tidak cukup hanya membuat program yang berjalan dengan benar, tetapi juga harus memperhatikan efisiensi dari program tersebut. Efisiensi ini menjadi sangat penting terutama ketika program digunakan untuk mengolah data dalam jumlah besar. Algoritma yang terlihat sederhana pada data kecil bisa menjadi sangat lambat dan tidak efisien ketika digunakan pada data yang besar.

Analisis kompleksitas algoritma hadir sebagai solusi untuk mengukur seberapa efisien suatu algoritma dalam hal waktu eksekusi dan penggunaan memori. Dengan menggunakan konsep Big O Notation, seorang programmer dapat memperkirakan performa algoritma tanpa harus bergantung pada spesifikasi perangkat keras. Hal ini membantu dalam memilih algoritma yang paling optimal untuk suatu permasalahan.

Pada praktikum ini, dilakukan berbagai percobaan untuk memahami perbedaan kompleksitas seperti $O(1)$, $O(n)$, hingga $O(n^2)$. Selain itu, juga dipelajari bagaimana mengukur waktu eksekusi secara empiris menggunakan fungsi bawaan JavaScript serta memahami konsep space complexity dan teknik optimasi algoritma.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah dalam praktikum ini adalah:

1. Mengapa analisis kompleksitas algoritma penting dalam pemrograman?
2. Apa yang dimaksud dengan Time Complexity dan Space Complexity?
3. Bagaimana cara membaca dan memahami notasi Big O?
4. Bagaimana perbedaan performa antara algoritma dengan kompleksitas berbeda?
5. Bagaimana cara mengoptimasi algoritma agar lebih efisien?

1.3 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

- Memahami pentingnya analisis efisiensi algoritma
- Menjelaskan konsep Time Complexity dan Space Complexity
- Memahami dan menggunakan notasi Big O
- Menganalisis kompleksitas dari berbagai algoritma
- Membandingkan performa algoritma secara empiris
- Meningkatkan kemampuan dalam mengoptimasi kode

BAB II

LANDASAN TEORI

2.1 Pengertian Big O Notation

Big O Notation adalah cara untuk menggambarkan kompleksitas waktu (runtime) dari sebuah algoritma. Ini membantu kita dalam menganalisis dan membandingkan efisiensi algoritma berdasarkan pertumbuhan ukuran inputnya. Di bawah ini adalah beberapa contoh Big O Notation yang umum dalam bahasa pemrograman Java beserta penjelasannya:

1. $O(1)$ – Constant Time:

Algoritma yang memiliki kompleksitas waktu konstan memiliki waktu eksekusi yang tetap, tidak peduli berapa besar ukuran inputnya. Contohnya adalah mengakses elemen dalam array dengan indeks tertentu atau melakukan operasi matematika sederhana.

2. $O(n)$ – Linear Time:

Algoritma dengan kompleksitas waktu linier memiliki waktu eksekusi yang sebanding dengan ukuran inputnya. Contohnya adalah melakukan iterasi melalui array atau daftar dan melakukan operasi pada setiap elemen.

3. $O(n^2)$ – Quadratic Time:

Algoritma dengan kompleksitas waktu kuadratik memiliki waktu eksekusi yang berhubungan dengan kuadrat dari ukuran inputnya. Contohnya adalah melakukan nested loop untuk mengakses setiap pasangan elemen dalam dua array.

4. $O(\log n)$ – Logarithmic Time:

Algoritma dengan kompleksitas waktu logaritmik memiliki waktu eksekusi yang tumbuh secara logaritmik seiring dengan ukuran inputnya. Contohnya adalah pencarian biner di mana setiap langkah mengurangi setengah ukuran input yang tersisa.

5. $O(2^n)$ – Exponential Time

Exponential Time biasanya digunakan dalam situasi dimana kita tidak terlalu tahu terhadap permasalahan yang dihadapi, sehingga mengharuskan kita mencoba setiap kombinasi dan permutasi dari semua kemungkinan.

2.2 Time Complexity

Time Complexity adalah seberapa lama waktu yang diperlukan untuk menjalankan suatu algoritma.

Contoh:

- Fungsi dengan satu loop $\rightarrow O(n)$
- Nested loop $\rightarrow O(n^2)$
- Operasi langsung $\rightarrow O(1)$

Hasil praktikum menunjukkan bahwa:

- Algoritma $O(1)$ sangat cepat karena tidak tergantung input
- Algoritma $O(n)$ meningkat secara linear
- Algoritma $O(n^2)$ sangat lambat untuk data besar

2.3 Space Complexity

Space Complexity adalah seberapa besar memori yang kita gunakan untuk menjalankan suatu algoritma.

Contoh:

- Menggunakan variabel biasa $\rightarrow O(1)$
- Membuat array baru $\rightarrow O(n)$
- Rekursi $\rightarrow O(n)$ karena call stack

Praktikum menunjukkan bahwa algoritma yang lebih cepat kadang membutuhkan memori lebih besar (trade-off).

2.4 Perbandingan Algoritma

Dalam praktikum dilakukan pengujian menggunakan `Date.now()` untuk membandingkan performa algoritma.

Hasilnya:

- Algoritma sederhana bisa lebih cepat jika kompleksitasnya rendah
- Nested loop menghasilkan waktu paling lama
- Penggunaan struktur data seperti **Set** dan **Map** dapat meningkatkan performa

2.5 Optimasi dan Refactoring Algoritma

Optimasi algoritma merupakan proses untuk meningkatkan efisiensi kode tanpa mengubah hasil output. Salah satu cara yang umum dilakukan adalah dengan menghindari penggunaan nested loop dan menggantinya dengan struktur data yang lebih efisien.

Dalam praktikum, dilakukan refactoring dari algoritma yang menggunakan nested loop menjadi algoritma yang menggunakan Set atau Map. Hasilnya menunjukkan peningkatan performa yang signifikan.

Selain itu, optimasi juga dapat dilakukan dengan mengurangi operasi yang tidak perlu, menggunakan pendekatan yang lebih sederhana, serta memilih algoritma yang sesuai dengan kebutuhan.

Dengan melakukan optimasi, program tidak hanya menjadi lebih cepat tetapi juga lebih efisien dalam penggunaan sumber daya

BAB III

METODOLOGI PENELITIAN

3.1 Alat dan Bahan

Alat:

1. Komputer dan Laptop
2. Mouse dan Keyboard
3. Akses Internet

Bahan:

1. Git Bash
2. Visual Studio Code

3.2 Langkah-langkah praktikum Function dan Array

1. Buka aplikasi Gitbash
2. Ketik: `cd/d`
3. Masuk ke folder repository
`cd nama-repository-kamu`
4. Membuat folder praktikum:
`mkdir praktikum-5`
`cd praktikum-5`
5. Inisialisasi project:
`npm init -y`
beginilah tampilannya di gitbash

```
ASUS@LAPTOP-IAIMS2IO MINGW64 ~
$ cd / d
bash: cd: too many arguments

ASUS@LAPTOP-IAIMS2IO MINGW64 ~
$ cd "/d/praktikum strukturdataaalogitma/"

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalogitma
$ cd 2025573010087_strukturdataalogitma

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma (main)
$ mkdir praktikum-5

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma (main)
$ cd praktikum-5

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma/praktikum-5 (main)
$ npm init -y
wrote to D:\praktikum strukturdataaalogitma\2025573010087_strukturdataalogitma\
praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```


- Kemudian lanjut membuat file didalam dan pastikan berada di dalam folder praktikum-5
- Ketik: `touch 01-intro-kompleksitas.js`
- Buka file 01-intro-kompleksitas.js di VS Code kemudian ketik program dan jalankan

```

01-intro-kompleksitas.js > 01-ukurWaktu
1 function ukurWaktu(label, fn){
2   const mulai = Date.now();
3   fn();
4   const selesai = Date.now();
5   console.log(`${label}: ${(selesai - mulai) ms}`);
6 }
7 const N = 100_000; // ukuran data: 100 ribu
8 function jumlahkanLinear(n){
9   let total = 0;
10  for (let i = 1; i <= n; i++) total += i;
11  return total;
12 }
13 function jumlahkanRumus(n){
14   return (n * (n + 1)) / 2;
15 }
16 function caraPasangan(arr){
17   const pasangan = [];
18   for (let i = 0; i < arr.length; i++){
19     for (let j = i + 1; j < arr.length; j++){
20       if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
21     }
22   }
23   return pasangan;
24 }
25 const data = Array.from({length: 5000}, (_, i) => i);
26 console.log('=== Perbandingan Waktu Eksekusi ===');
27 ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
28 ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
29 ukurWaktu('O(n^2) cariPasangan ', () => caraPasangan(data));
30
31 console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
32

```

- Kemudian ketik: `node 01- intro-kompleksitas.js` agar hasil programnya muncul di Gitbash

```

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaa/goritma/2025573010087_s
strukturdataaa/goritma/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaa/goritma/2025573010087_s
strukturdataaa/goritma/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 2 ms
O(n^2) cariPasangan : 33 ms

Hasil sama? true

```

- Lanjutkan membuat file selanjutnya
Ketik: `touch 02-destructuring-spread.js`
- Buka file 02-destructuring-spread.js di VS Code kemudian ketik program dan jalankan

```

02-destructuring-spread.js >
1 function ambilPertama(arr) { return arr[0]; } // O(1)
2 function tambahkanItem(arr, item) { arr.push(item); } //O(1)
3 function isGenap(n) { return n % 2 === 0; } //O(1)
4
5 function binarySearch(arr, target){
6   let kiri = 0, kanan = arr.length - 1;
7   let langkah = 0;
8   while(kiri <= kanan){
9     langkah++;
10    const tengah = Math.floor((kiri + kanan) / 2);
11    if (arr[tengah] === target){
12      console.log('Ditemukan di indeks $(tengah) setelah $(langkah) langkah');
13      return tengah;
14    }
15    if (arr[tengah] < target) kiri = tengah + 1;
16    else kanan = tengah - 1;
17  }
18  return -1;
19 }
20
21 function cariMax(arr){
22   let maks = arr[0];
23   for (const x of arr) if (x > maks) maks = x;
24   return maks;
25 }
26
27 function bubbleSortNaif(arr){
28   const a = [...arr];
29   for (let i = 0; i < a.length; i++)
30     for (let j = 0; j < a.length - i - 1; j++)
31       if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
32   return a;
33 }
34
35 function fibRekursif(n){
36   if (n <= 1) return n;
37   return fibRekursif(n-1) + fibRekursif(n-2); // dua panggilan rekursif
38 }
39 console.log('=== O(1) - selalu cepat ===');
40 console.log(ambilPertama([10,20,30,40,50])); //10
41 console.log(isGenap(42)); //true
42 console.log('===== O(n) a) - Bineka: Kanan a) =====');
43 function binarySearch(arr: any, target: any): number {
44   binarySearch(sorted, 731,452); //cari di 1juta data
45 }
46 console.log('===== O(n) - Linear Search ===');
47 console.log('Max dari 1000 elemen: ',
48   cariMax(Array.from({length:1000},()=> Math.random()*10000)));
49
50 console.log('===== O(n^2) - Bubble Sort (kecil saja) ===');
51 console.log(bubbleSortNaif([64,34,25,12,22,11,90]));
52
53 console.log('===== O(2^n) - Fibonacci Rekursif (n kecil) ===');
54 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
55 console.log('');
56 console.log('Waktu fib(35) O(2^n):');
57 let t = Date.now(), fibRekursif(35); console.log(Date.now()-t, 'ms');
58
59 console.log('Waktu fib(35) O(n) memoization:');
60 const memo = {};
61 function fibMemo(n){
62   if(n <= 1) return n;
63   if (memo[n]) return memo[n];
64   return memo[n] = fibMemo(n-1) + fibMemo(n-2);
65 }
66 t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');

```

- Kemudian ketik: `node 02-destructuring-spread.js` agar hasil programnya muncul di GitBash

```

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_s
strukturdataaalgorithm/praktikum-5 (main)
$ touch 02-destructuring-spread.js

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_s
strukturdataaalgorithm/praktikum-5 (main)
$ node 02-destructuring-spread.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 999

=== O(n^2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) - Fibonacci Rekursif (n kecil!) ===
011235813213455

Waktu fib(35) O(2^n):
199 ms
Waktu fib(35) O(n) memoization:
1 ms

```

13. Lanjutkan ketik: [touch 03-space-complexity.js](#)

14. Buka file 03-space-complexity.js di VS Code kemudian ketik program dan jalankan

```

JS 03-space-complexity.js > hitungUnik
1  function jumlahArray(arr){
2      let total = 0;
3      for (const x of arr) total += x;
4      return total;
5  }
6  function duplikasiArray(arr){
7      const baru = [];
8      for (const x of arr) baru.push(x * 2);
9      return baru;
10 }
11 function faktorialRekursif(n){
12     if (n <= 1) return 1;
13     return n * faktorialRekursif(n - 1); // n frame di call stack
14 }
15 function faktorialIteratif(n){
16     let hasil = 1;
17     for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
18     return hasil;
19 }
20 const arr = [1,2,3,4,5,6,7,8,9,10];
21
22 console.log('Jumlah array      : ', jumlahArray(arr));
23 console.log('Duplikasi array   : ', duplikasiArray(arr));
24 console.log('Faktorial 10 rekursif : ', faktorialRekursif(10));
25 console.log('Faktorial 10 iteratif : ', faktorialIteratif(10));
26
27 function hitungUnik(arr) {
28     const seen = new Set(); // Set tumbuh hingga n elemen
29     for (const x of arr) seen.add(x);
30     return
31 }
32 const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
33 console.log('Elemen unik:', hitungUnik(dataAcak));

```

15. Ketik `node 03- space-complexity.js` agar hasil program muncul di Git Bash

```

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_s
strukturdataaalgorithm/praktikum-5 (main)
$ touch 03-space-complexity.js

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_s
strukturdataaalgorithm/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array      : 55
Duplikasi array   : [
  2,  4,  6,  8, 10,
 12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: undefined

```

16. Lanjutkan: [touch 04-refactor.js](#)

17. Buka file 04-refactor.js di VS Code kemudian ketik program dan jalankan

```
JS 04-refactor.js > ...
1 function adaDuplikatLambat(arr){
2 }
3
4 function adaDuplikatCepat(arr){
5     const seen = new Set();
6     for (const x of arr){
7         if (seen.has(x)) return true;
8         seen.add(x);
9     }
10    return false;
11 }
12
13 function frekuensiLambat(arr){
14     const unik = [];
15     const hitung = [];
16     for (const x of arr){
17         const idx = unik.indexOf(x);
18         if (idx === -1){ unik.push(x); }
19     }
20    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
21 }
22
23 function frekuensiCepat(arr){
24     const counter = {};
25     for (const x of arr) counter[x] = (counter[x] || 0) + 1;
26    return counter;
27 }
28
29 const besar = Array.from({length: 20000}, () => Math.floor(Math.random()* 1000));
30 let t = Date.now();
31 adaDuplikatLambat(besar);
32
33 console.log('Duplikat LAMBAT O(n^2):', Date.now()-t, 'ms');
34 t = Date.now();
35 adaDuplikatCepat(besar);
36 console.log('Duplikat CEPAT O(n) :', Date.now()-t, 'ms');
37 const kata = ['a','b','a','c','b','a','d'];
38 console.log('\nFrekuensi:', frekuensiCepat(kata));
```

18. Ketik node 04- refactor.js agar hasil program muncul di Git Bash

```
ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_strukturdataalgorithm/praktikum-5 (main)
$ touch 04-refactor.js

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_strukturdataalgorithm/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n^2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

19. Kemudian buat folder tugas di Git Bash

mkdir tugas

touch tugas/tugas-1.js tugas-2.js

```
ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_strukturdataalgorithm/praktikum-4 (main)
$ mkdir tugas

ASUS@LAPTOP-IAIMS2IO MINGW64 /d/praktikum strukturdataaalgorithm/2025573010087_strukturdataalgorithm/praktikum-4 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

20. Kerjakan tugas di folder tersebut

3.3 Hasil Praktikum

- Latihan 1

```
JS latihan-1.js > ...
1  function hitungKompleksitas(n,fn){
2      const mulai = Date.now();
3      fn(n);
4      const selesai = Date.now();
5      console.log(`waktu: ${selesai - mulai} ms`);
6  }
7  function fungsiA(n){
8      return n * 2;
9  }
10 function fungsiB(n){
11     for(let i = 0; i < n; i++)
12         for(let j = 0; j < n; j++)
13             console.log(i,j);
14 }
15 function fungsiC(n){
16     for(let i = 1; i < n; i *= 2)
17         console.log(i)
18 }
19 function fungsiD(n){
20     arr.forEach(x => {
21         arr.forEach(y => {
22             arr.forEach(z => {
23                 console.log(x,y,z)
24             });
25         });
26     });
27 }
28
29 const n = 1000;
30
31 console.log("fungsi A (0(1))");
32 hitungKompleksitas(n,fungsiA);
33
34 console.log("fungsi B (0(n^2))");
35 hitungKompleksitas(n,fungsiB);
36
37 console.log("fungsi C (0(log n))");
38 hitungKompleksitas(n,fungsiC);
39
40 console.log("fungsi D (0(n^3))");
41 hitungKompleksitas(n,fungsiD);
```

- Latihan 2

```
JS caripasangan.js > ...
41 function cariPasanganLambat(arr, target) {
42     for (let i = 0; i < arr.length; i++) {
43         for (let j = i + 1; j < arr.length; j++) {
44             if (arr[i] + arr[j] === target) {
45                 return [arr[i], arr[j]];
46             }
47         }
48     }
49     return null;
50 }
51
52 function cariPasanganCepat(arr, target) {
53     const seen = new Set();
54
55     for (let num of arr) {
56         const pasangan = target - num;
57
58         if (seen.has(pasangan)) {
59             return [pasangan, num];
60         }
61         seen.add(num);
62     }
63
64     return null;
65 }
66
67 const arr1 = [2, 7, 11, 15];
68 const target = 9;
69
70 console.log('=== Test Kecil ===');
71 console.log('Lambat:', cariPasanganLambat(arr1, target));
72 console.log('Cepat:', cariPasanganCepat(arr1, target));
73
74
75 function generateArray(n) {
76     const hasil = [];
77     for (let i = 0; i < n; i++) {
78         hasil.push(Math.floor(Math.random() * 50000));
79     }
80     return hasil;
81 }
82
83
84 function ukurWaktu(label, fn, arr, target) {
85     const mulai = Date.now();
86     fn(arr, target);
87     const selesai = Date.now();
88     console.log(`${label}: ${selesai - mulai} ms`);
89 }
90
91 const arrBesar = generateArray(50000);
92 const targetBesar = -1;
93
94 console.log(`\n=== Test Besar (50.000 data) ===`);
95 ukurWaktu('Lambat', cariPasanganLambat, arrBesar, targetBesar);
96 ukurWaktu('Cepat', cariPasanganCepat, arrBesar, targetBesar);
```

3.4 submit ke Github

1. Ketik: `cd ..`
2. Tambahkan folder
`Git add praktikum-5/`
3. Commit ke push:

Ketik: git commit -m"Praktikum 5: Analisis Kompleksitas Algoritma "

git push origin main

```
ASUSLAPTOP-IAIMS210 MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma/praktikum-5 (main)
$ cd ..

ASUSLAPTOP-IAIMS210 MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it

ASUSLAPTOP-IAIMS210 MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main c89f067] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 451 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-destructuring-spread.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

ASUSLAPTOP-IAIMS210 MINGW64 /d/praktikum strukturdataaalogitma/2025573010087_s
strukturdataalogitma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.86 KiB | 830.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/nauranadhifa24/2025573010087_strukturdataalogitma.git
 22a5c12..c89f067  main -> main
```

4. Selanjutnya, buka Github melalui browser dan masuk ke bagian repository. Jika proses berhasil, maka tampilannya akan terlihat seperti berikut

```
▼ folder praktikum-5
  + 01-intro-kompleksitas.js
  + 02-destructuring-spread.js
  + 03-space-complexity.js
  + 04-refactor.js
  + latihan-1.js
  + package.json
▼ folder tugas
  + tugas-1.js
  + tugas-2.js
```

3.5 Analisis Hasil

Berdasarkan praktikum yang dilakukan, terlihat bahwa algoritma dengan kompleksitas kecil seperti **O(1)** dan **O(n)** memiliki performa yang lebih cepat dibandingkan **O(n²)**, terutama saat data semakin besar. Pengujian menggunakan Date.now() menunjukkan bahwa semakin tinggi kompleksitas, semakin lama waktu eksekusinya.

Selain itu, penggunaan struktur data seperti **Set** dapat meningkatkan kecepatan algoritma dari **O(n²)** menjadi **O(n)**, meskipun membutuhkan memori tambahan. Hal ini menunjukkan adanya trade-off antara waktu dan ruang.

Secara keseluruhan, praktikum ini membuktikan bahwa pemilihan algoritma dan struktur data yang tepat sangat penting untuk menghasilkan program yang efisien.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting dalam menentukan efisiensi suatu program. Dengan menggunakan Big O Notation, kita dapat memahami bagaimana performa algoritma berubah seiring bertambahnya jumlah data. Hasil praktikum menunjukkan bahwa algoritma dengan kompleksitas kecil seperti $O(1)$ dan $O(n)$ jauh lebih efisien dibandingkan $O(n^2)$, terutama pada data berukuran besar. Selain itu, dua algoritma yang menghasilkan output yang sama dapat memiliki perbedaan waktu eksekusi yang sangat signifikan.

Selain itu, tidak hanya waktu yang perlu diperhatikan, tetapi juga penggunaan memori atau space complexity. Terdapat hubungan trade-off antara waktu dan ruang, di mana algoritma yang lebih cepat seringkali membutuhkan memori lebih besar. Oleh karena itu, pemilihan algoritma harus disesuaikan dengan kebutuhan dan kondisi yang ada. Penggunaan struktur data yang tepat seperti Set atau Map juga terbukti dapat meningkatkan performa algoritma secara signifikan.

4.2 Saran

Dalam pengembangan program, sebaiknya programmer tidak hanya fokus pada hasil yang benar, tetapi juga memperhatikan efisiensi algoritma yang digunakan. Mahasiswa disarankan untuk lebih sering berlatih menganalisis kompleksitas dan memahami pola-pola Big O agar dapat memilih solusi yang paling optimal. Selain itu, praktik langsung dan eksperimen sangat penting untuk memahami perbedaan performa algoritma secara nyata. Dengan demikian, diharapkan mahasiswa mampu mengembangkan program yang tidak hanya berjalan dengan baik, tetapi juga efisien dan optimal.